

## COURSE NAME: THEORY OF COMPUTATION

COURSE CODE: CSA1304

CO3 – AT1 : Case Study

### 1. Designing a Grammar for Arithmetic Expressions

A compiler team's calculator parser supports  $+$ ,  $-$ ,  $\times$ ,  $\div$  and parentheses. The naive grammar  $E \rightarrow E+E \mid E-E \mid E * E \mid E / E \mid (E) \mid \text{id}$  is ambiguous — it gives two different parse trees for  $a+b*c$  (one that groups  $(a+b)*c$  and one that groups  $a+(b*c)$ ), and it has no notion of which operator binds tighter.

#### Optimized, unambiguous CFG

Ambiguity and precedence are fixed together by layering the grammar into one non-terminal per precedence level, with the highest-precedence operators nearest the leaves:

$E \rightarrow E + T \mid E - T \mid T$  (lowest precedence:  $+$  and  $-$ )

$T \rightarrow T * F \mid T / F \mid F$  (next precedence:  $*$  and  $/$ )

$F \rightarrow ( E ) \mid \text{id}$  (highest precedence: atoms / parentheses)

Left recursion in  $E$  and  $T$  ( $E \rightarrow E+T$ ,  $T \rightarrow T * F$ ) automatically makes  $+$ ,  $-$ ,  $*$ ,  $/$  left-associative, matching normal calculator semantics ( $a-b-c$  means  $(a-b)-c$ ).

#### Parse tree for $a + b * c$

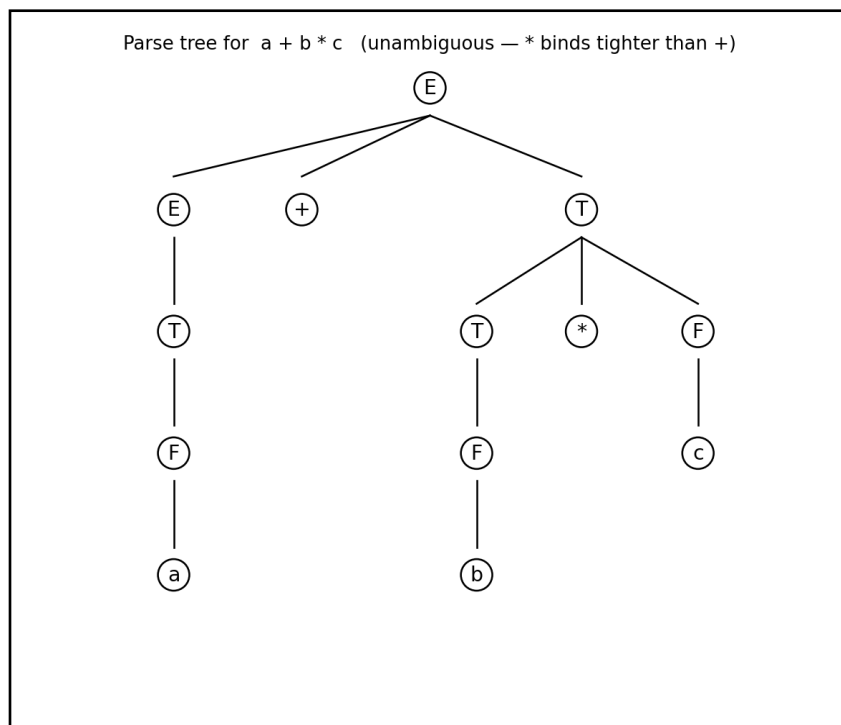


Fig. 1 —  $a + b * c$  parses uniquely as  $a + (b * c)$

### Left-recursion-eliminated form (for a top-down / recursive-descent parser)

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid - T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid / F T' \mid \epsilon$

$F \rightarrow ( E ) \mid \text{id}$

### How this removes ambiguity and improves parsing

- Exactly one non-terminal exists per precedence level, so a  $*$ ,  $/$ -expression can only ever appear nested inside a  $+$ ,  $-$  expression and never the other way round — this is what forces  $b*c$  to bind tighter than  $a+$  and eliminates the second, incorrect parse tree.
- Each  $E$  and  $T$  is left-recursive, so repeated operators of the same precedence ( $a+b+c$ ,  $a*b*c$ ) are forced to associate to the left, matching standard evaluation order — no derivation can produce right-associative grouping.
- The grammar is now unambiguous: for every string in the language there is exactly one parse tree, so a parser needs no error-prone tie-breaking rules.
- After eliminating left recursion ( $E'$ ,  $T'$  forms), the grammar becomes suitable for an  $LL(1)$  / recursive-descent parser — parsing runs in linear time with a single left-to-right pass and one token of lookahead, which is far more efficient than a backtracking parser needed for the ambiguous version.

## 2. Mini Programming Language Parser Development

An IoT scripting language needs variable declarations, conditionals, and loops. The original grammar reports "syntax conflicts" — typically the symptom of left recursion (which breaks top-down parsers) and/or overlapping first-symbols between alternatives (which breaks  $LL(1)$  lookahead).

### Optimized CFG for the language constructs

$\text{Program} \rightarrow \text{Stmt Program} \mid \epsilon$

$\text{Stmt} \rightarrow \text{Decl} \mid \text{Assign} \mid \text{IfStmt} \mid \text{WhileStmt}$

$\text{Decl} \rightarrow \text{var id} ;$

$\text{Assign} \rightarrow \text{id} = \text{Expr} ;$

$\text{IfStmt} \rightarrow \text{if} ( \text{Cond} ) \{ \text{Program} \} \text{ElsePart}$

$\text{ElsePart} \rightarrow \text{else} \{ \text{Program} \} \mid \epsilon$

$\text{WhileStmt} \rightarrow \text{while} ( \text{Cond} ) \{ \text{Program} \}$

$\text{Cond} \rightarrow \text{Expr RelOp Expr}$

$\text{RelOp} \rightarrow < \mid > \mid == \mid !=$

$\text{Expr} \rightarrow \text{Term Expr}'$

$\text{Expr}' \rightarrow + \text{Term Expr}' \mid - \text{Term Expr}' \mid \epsilon$

$\text{Term} \rightarrow \text{id} \mid \text{num}$

## Removing left recursion and reducing conflicts

- Expr was originally left-recursive ( $\text{Expr} \rightarrow \text{Expr} + \text{Term}$ ); rewritten with the standard transformation  $\text{Expr} \rightarrow \text{Term Expr}'$ ,  $\text{Expr}' \rightarrow +\text{Term Expr}' \mid -\text{Term Expr}' \mid \varepsilon$ , which removes the infinite left-recursive loop a top-down parser would otherwise fall into.
- Every alternative under Stmt starts with a distinct keyword or token — var, id, if, while — so a parser needs only one token of lookahead to choose the right production (no two alternatives share a common prefix, so no left-factoring is required at that level).
- The dangling-else ambiguity is resolved structurally: ElsePart is a single optional non-terminal attached to exactly one IfStmt production, so "else" always binds to the nearest unmatched "if" by construction, instead of being offered as a separate competing top-level rule.
- $\text{Program} \rightarrow \text{Stmt Program} \mid \varepsilon$  is right-recursive (not left-recursive), so statement sequences can be parsed straightforwardly by a loop/recursive-descent routine without special handling.

## Why the optimized grammar parses more efficiently

- With left recursion removed, a recursive-descent (LL(1)) parser can process the grammar in a single left-to-right scan with no backtracking — parsing time is linear in the length of the program,  $O(n)$ .
- Because every production choice is uniquely determined by the next token, the parser never needs to explore and discard alternatives, which is what caused the original "syntax conflicts".
- Cleanly separated non-terminals (Decl, Assign, IfStmt, WhileStmt) make the parser's code modular — each corresponds to one parsing function, which is easy to extend with new statement types.

## 3. Grammar Optimization for SQL Query Validation

A validation tool for simplified SELECT / FROM / WHERE queries has redundant productions — typically a grammar with separate top-level rules for "query with WHERE" and "query without WHERE", and separate rules for a 1-column list vs a multi-column list, which duplicates work during parsing.

### Optimized CFG

Query  $\rightarrow$  SELECT ColList FROM id WhereClause ;

WhereClause  $\rightarrow$  WHERE Cond  $\mid \varepsilon$

ColList → id ColListTail

ColListTail → , id ColListTail | ε

Cond → id RelOp Value | Cond AND Cond | Cond OR Cond

RelOp → = | < | > | <= | >= | <>

Value → id | num | string

### What was optimized, and how

- The WHERE clause is factored into a single optional non-terminal WhereClause → WHERE Cond | ε, replacing two duplicated top-level Query productions (one for each case) with one production that always looks the same up to that point — this removes the redundant production and lets the parser commit to Query → SELECT ColList FROM id ... immediately, only branching on WHERE vs. ; afterwards.
- The column list uses right recursion via ColListTail instead of a naive left-recursive or doubly-duplicated rule ("one column" vs "column, more columns" as separate top-level alternatives) — now there is exactly one systematic way to parse any number of columns, including exactly one.
- Every remaining alternative begins with a distinct terminal (SELECT, WHERE / ε, comma / ε), so the grammar is LL(1)-parseable with a single token of lookahead and no ambiguity between rules.

### Why this speeds up syntax analysis

- Removing duplicate productions shrinks the parsing table (fewer entries to check per token), which directly reduces per-token decision time.
- A deterministic, non-redundant grammar lets the parser make one pass with no backtracking, giving linear-time O(n) validation instead of potentially exploring multiple redundant derivations for the same query.
- Because the grammar is unambiguous, each valid SQL string has exactly one parse tree — validation and any downstream processing (e.g. building a query plan) can proceed directly from that single parse tree.

## 4. Natural Language Processing Chatbot

The chatbot only needs to accept simple Subject–Verb–Object sentences (e.g. "Students learn TOC"). The initial CFG incorrectly accepts invalid sentences — a common cause is a single generic "Word" non-terminal used in every slot, so any word can appear in any position (e.g. "Learn Students TOC" or "TOC TOC TOC" would wrongly be accepted).

### Optimized CFG

$S \rightarrow NP VP$

$NP \rightarrow Det N \mid N$

$VP \rightarrow V NP$

$Det \rightarrow the \mid a$

$N \rightarrow Students \mid TOC \mid teacher \mid program \mid computer \mid \dots$

$V \rightarrow learn \mid study \mid teaches \mid runs \mid writes \mid \dots$

Example derivation:  $S \Rightarrow NP VP \Rightarrow N VP \Rightarrow Students VP \Rightarrow Students V NP \Rightarrow Students learn$

$NP \Rightarrow Students learn N \Rightarrow Students learn TOC$ .

### How this fixes validation accuracy

- Words are partitioned into separate part-of-speech non-terminals (N, V, Det) instead of one shared "any word, anywhere" category — a noun can only be generated where NP expects it, and a verb only where VP expects it, so a sentence with the wrong word order (verb where a noun is expected, etc.) has no valid derivation and is correctly rejected.
- The single top-level rule  $S \rightarrow NP VP \rightarrow NP V NP$  structurally enforces exactly the Subject–Verb–Object pattern; sentences that skip the object, skip the verb, or repeat a category out of order cannot be derived at all, since there is no  $\epsilon$ -production anywhere that would let a required slot be silently omitted.
- Restricting each terminal to one specific non-terminal's alternatives (a finite lexicon per category) prevents accidental over-generation — the grammar only ever accepts strings whose words are used in a role consistent with their part of speech.

## 5. Parser Design for HTML-like Markup Language

A markup-language parser must validate that opening and closing tags are properly nested, e.g. `<div><p>text</p></div>` is valid but `<div><p>text</div></p>` is not. This kind of "matching, correctly nested" structure is exactly the classical Dyck / balanced-bracket language, which is context-free but not regular (a finite automaton cannot count unbounded nesting depth; a grammar with a stack, i.e. a PDA/CFG, can).

### Optimized CFG (finite, known set of tag names)

```
Doc    → Element Doc | ε
Element → <div> Doc </div>
        | <p>  Doc </p>
        | <span> Doc </span>
        | text
```

This is structurally identical to the multi-bracket-type balanced language  $S \rightarrow (S) S \mid [S] S \mid \{S\} S \mid \epsilon$  — each tag type gets its own production whose opening and closing halves are hard-wired together in the same rule, so a `</p>` can only ever close a `<p>` that was opened by that same production instance, never a `<div>`.

### Parse tree example

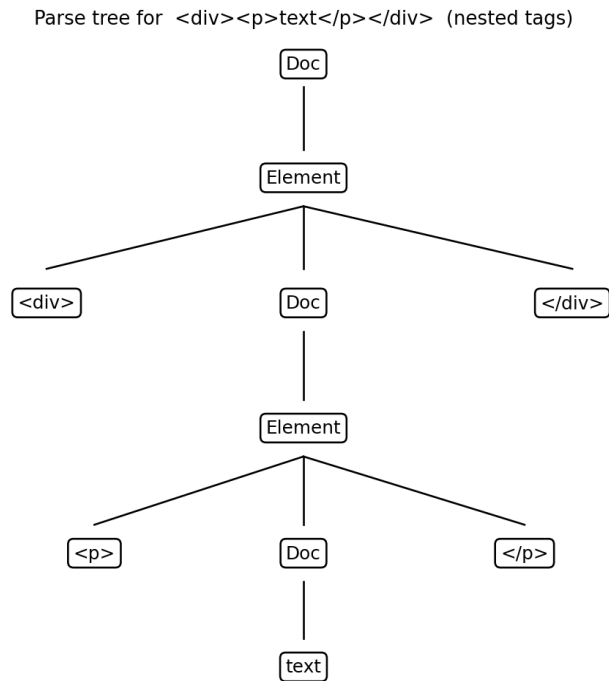


Fig. 2 — `<div><p>text</p></div>` parses with correctly matched, nested tags

## How this supports recursive nesting and improves reliability

- `Doc`  $\rightarrow$  `Element Doc` allows any number of sibling elements in sequence, while `Element`  $\rightarrow$  `<tag> Doc </tag>` allows arbitrarily deep nesting inside a tag — because `Doc` appears again inside `Element`'s own definition, the grammar is recursive and can generate nesting of unbounded depth, which a regular grammar/finite automaton fundamentally cannot do (it would need infinitely many states to remember arbitrarily deep unmatched-tag counts).
- Because each opening tag's production is the only place its matching closing tag can appear, a derivation exists only when every opening tag is closed by the same tag type in the correct (LIFO / stack-like) order — this is precisely what "properly nested" requires, and it rules out cross-nested strings like `<div><p>text</div></p>` since no production can generate a `</div>` immediately after an unclosed `<p>`.
- Adding a new tag type only requires adding one new `Element` alternative, without touching the recursive `Doc` definition — this keeps the grammar easy to extend and the parser's tag-matching logic simple and reliable (each recursive-descent call for `Element` knows exactly which closing tag to expect).